

Project on

Physical Design Implementation of Serial Peripheral Interface Using Qflow

Name : G.Chandrasekhara sai

Course : Maven silicon Physical Design internship

Batch : B2B PDI 01

TABLE OF CONTENTS

1. Introduction
2. Serial Peripheral Interface
3. Invoke the tool
4. Synthesis
5. Placement
6. Static Timing Analysis
7. Routing
8. DRC (design Rule Check)
9. LVS
10. GDS
11. Final Output

Introduction

Qflow –An Open Source Digital Synthesis flow

A digital synthesis flow is a set of tools and methods used to turn a circuit design written in a high-level behavioral language like verilog or VHDL into a physical circuit, which can either be configuration code for an FPGA target like a Xilinx or Altera chip, or a layout in a specific fabrication process technology, that would become part of a fabricated circuit chip. Several digital synthesis flows targeting FPGAs are available, usually from the FPGA manufacturers, and while they are typically not open source, they are generally distributed for free (presumably on the optimistic assumption that more people will be buying more FPGA hardware).

Qflow is a complete tool chain for synthesizing digital circuits starting from verilog source and ending in physical layout for a specific target fabrication process. Simple enough to be used by students in beginnerlevel digital logic courses, yet powerful enough to be included into large EDA systems like Cadence or Synopsys, commercial electronics designers and researchers in cutting-edge environments have relied on Qflow for several years to maintain quality and low hardware costs. Qflow is an open source tool that has been used in digital circuit and system design classes at universities and colleges around the world, and some small businesses and individuals also use it as part of inhouse silicon design. The algorithms for digital design flow are still locked up in closed source software, and development is hard outside the domain of the big EDA tools vendors. Qflow is an excellent example of how, when the best components that are available are open-source, and that can be maintained by the user within the flow, then better alternatives to any component exist.

Qflow consists of some list of tools like:

- Yosys – RTL Synthesis
- blif2struct – High Format Net Synthesis
- graywolf – Placement
- magic – VLSI Layout Tool
- netgen – LVS Layout vs Schematic
- OpenSTA – Static Timing Analysis tool

Serial Peripheral Interface:

Serial Peripheral Interface, or SPI, is a very common communication protocol used for two-way communication between two devices. A standard SPI bus consists of 4 signals:

- MOSI (Master Out Slave In) MISO (Master In Slave Out)
- SCLK (Serial Clock)
- SS (Slave Select)

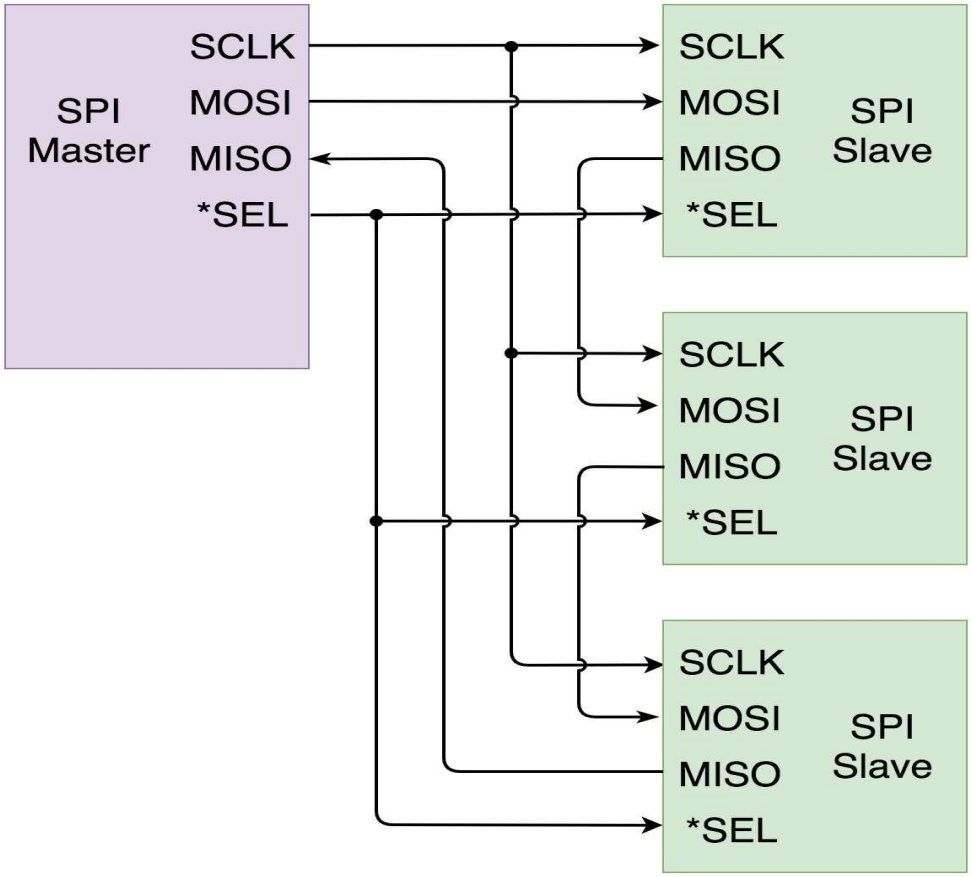
Unlike an asynchronous serial interface, SPI is not convenient. An SPI bus has one master and one or more slaves. The master can talk to only one slave on the bus, but each slave can only talk to the master. Each slave on the bus must have its own unique slave select signal. The master uses the slave select signals to select which slave it will be talking to. Since SPI also includes a clock signal, both devices do not need to agree on a data transfer baud rate. The only requirement is that the clock is lower than the maximum frequency for all devices connected.

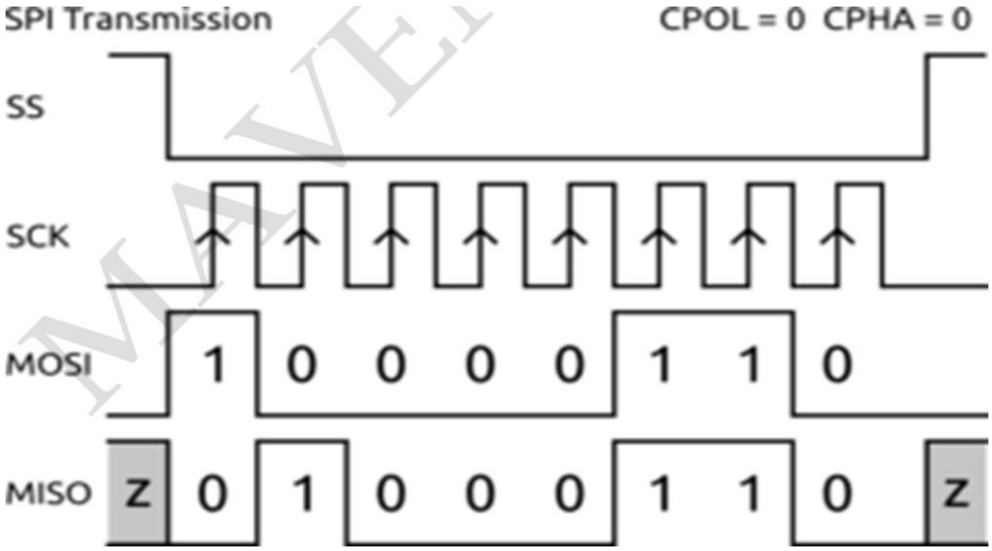
There are 4 different SPI bus standards that all have to do with the SCK signal.
The 4 modes are broken down into two parameters, CPOL and CPHA.

- CPOL stands for Clock Polarity and designates the default value (high/low) of the SCK signal when the line is idle.
- CPHA stands for Clock Phase and determines which edge of the clock signals data capture (rising/falling).

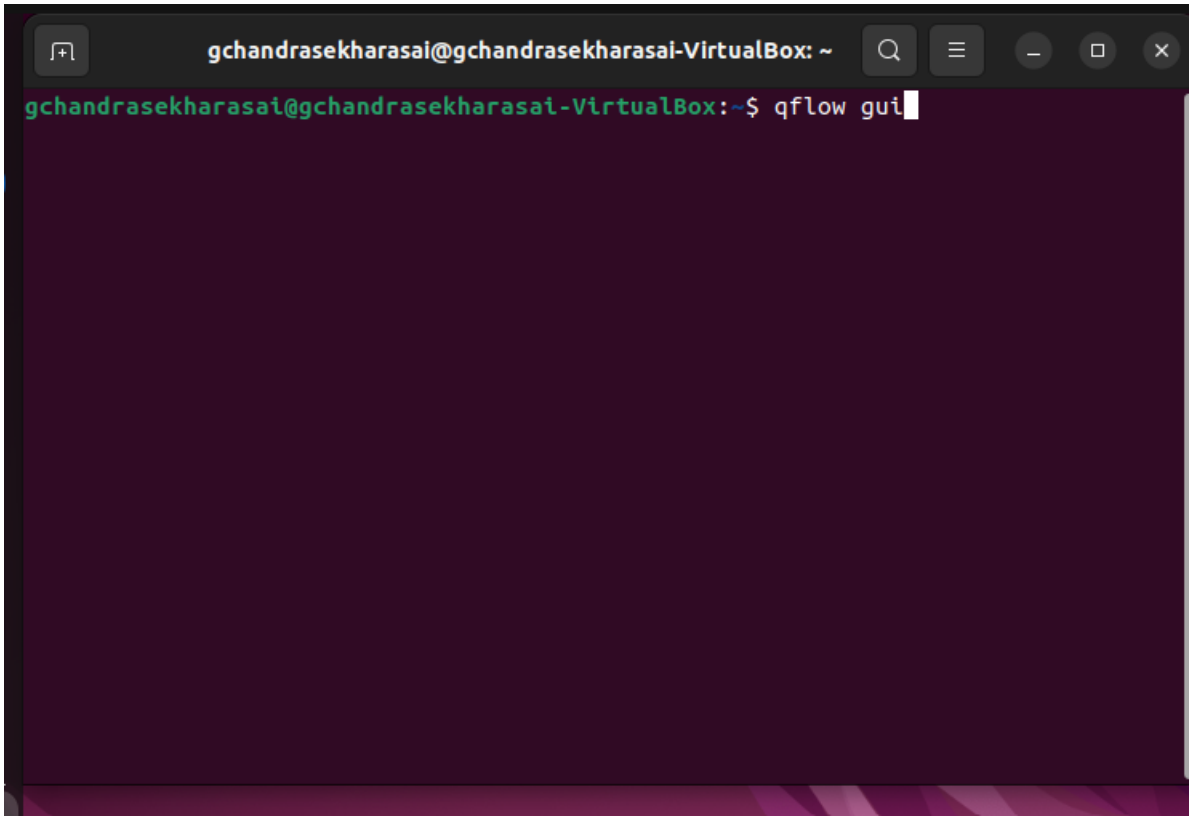
The slave or device must specify these parameters so you can adjust accordingly. The most common settings are CPOL=0 (idle low) and CPHA=0 (sample rising edge).

A diagram will help you understand the following in a better way



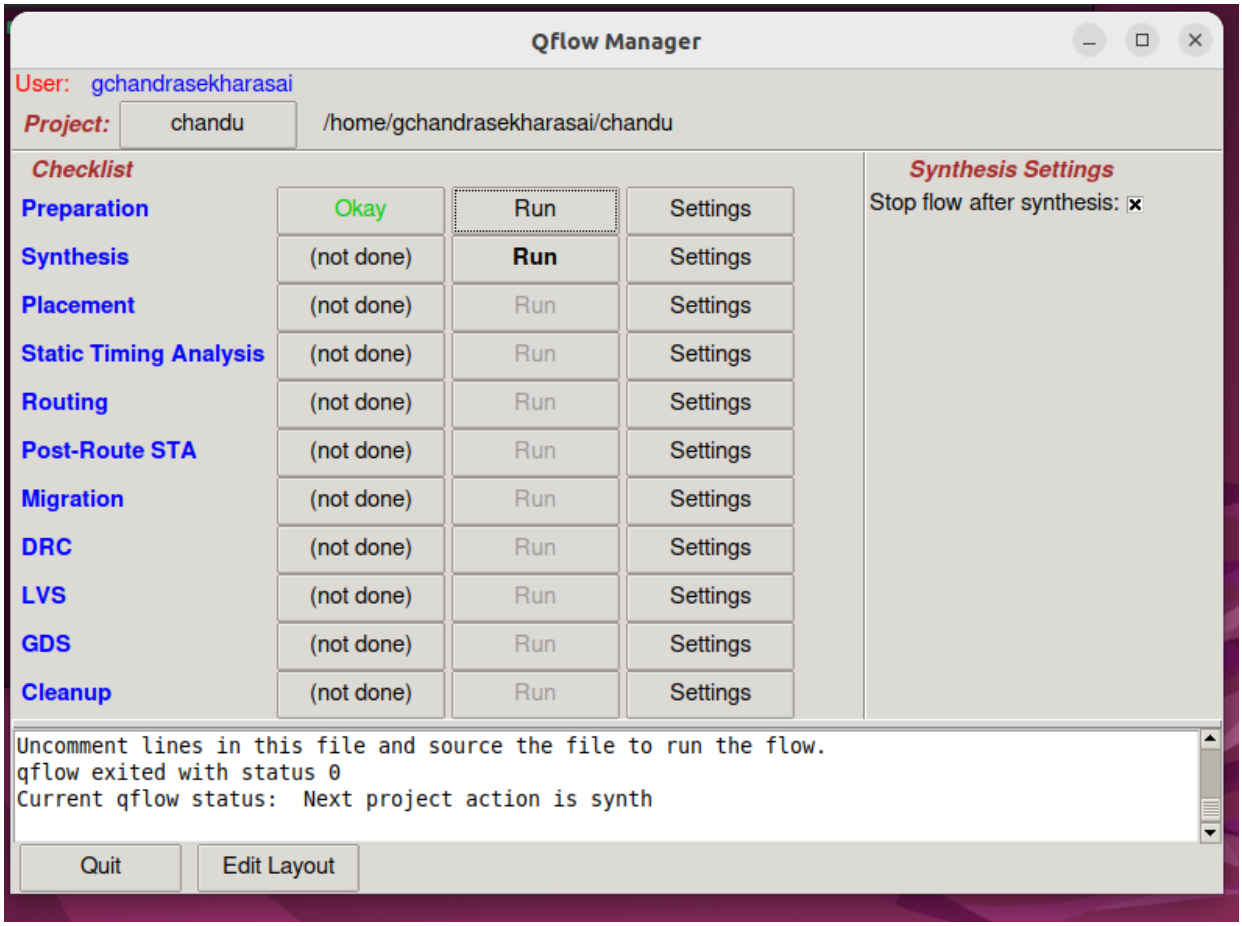


Invoke The Tool ❖ To invoke the tool we required Terminal Box After opening the terminal box we have to type qflow gui and press enter. It invokes the Qflow tool interface.



Qflow Gui:

Qflow GUI is a graphical user interface designed to work with the Qflow EDA (Electronic Design Automation) tools suite. It provides a user-friendly layer for interacting with Qflow's tools and processes, aiming to simplify the workflow of designing digital circuits and systems. The GUI is an extension of the powerful features of the Qflow toolchain more accessible to users who prefer visual interactions over command-line operations.



Using the Qflow GUI Tool

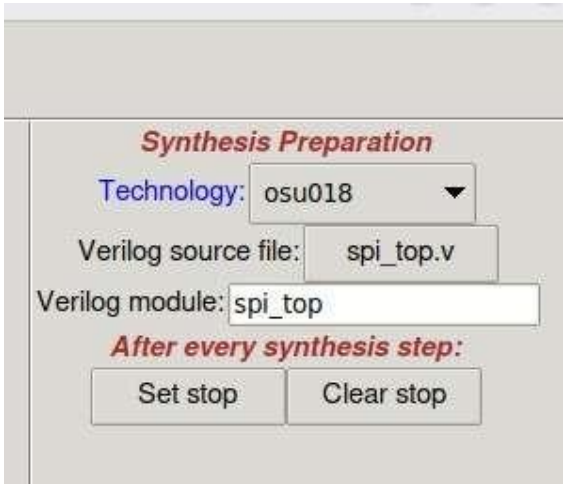
The Qflow GUI is a graphical interface that helps us run all the important steps involved in digital chip design, especially when working on physical design. Instead of typing commands one by one in the terminal, the Qflow GUI gives us buttons and menus to work with, which is easier for many users.

Once we open the Qflow GUI, we'll see a list of steps (or tools) that need to be run in a specific order — like synthesis, placement, routing, etc. Each tool must be run one after the other. We can do this by clicking on the "Run" button next to each step.

If the tool runs successfully, it will show a message saying "OKAY" in the status column.
If something goes wrong, it will show "FAIL" instead. This means we need to check what went wrong, fix the issue (usually a syntax error or a missing file), and try running it again.

Before Running Tools

Before we start running any tool in Qflow, there are a few setup steps we must complete:

1. Select the Technology File
    - This file contains all the physical and electrical rules of the chip (like dimensions, layers, spacing, etc.). o You'll see names like osu018 or osu035 — choose the one that matches your design process.
2. Choose the Top Module of Your Verilog File
    - Your design will be written in Verilog (.v file). You need to tell Qflow which is the top-level module (the main circuit you're working with). o Make sure your Verilog code is error-free. If there are syntax mistakes, fix them first.
    - o 

3. After selecting the technology and Verilog file, you're ready to begin.

- Start by clicking "Run Preparation".
- Once you run this step, Qflow will automatically create several folders inside your project directory. These folders include:
    - layout/ – for layout files o source/ – your

    input Verilog files o synthesis/ – synthesis reports

    - log/ – log files from each step o tmp/ or tech/

    – temporary files or tech-specific data

These folders help organize your project and make it easy to debug if something fails.

Synthesis

Synthesis is a key part of the digital design process. It's the step where high-level code (written in languages like Verilog or VHDL) is turned into a gate-level version that can be built into actual hardware, such as FPGAs or ASICs. In Qflow, this step is done using a tool called Yosys, and the Qflow GUI makes it easier by giving a visual interface to run and manage it.

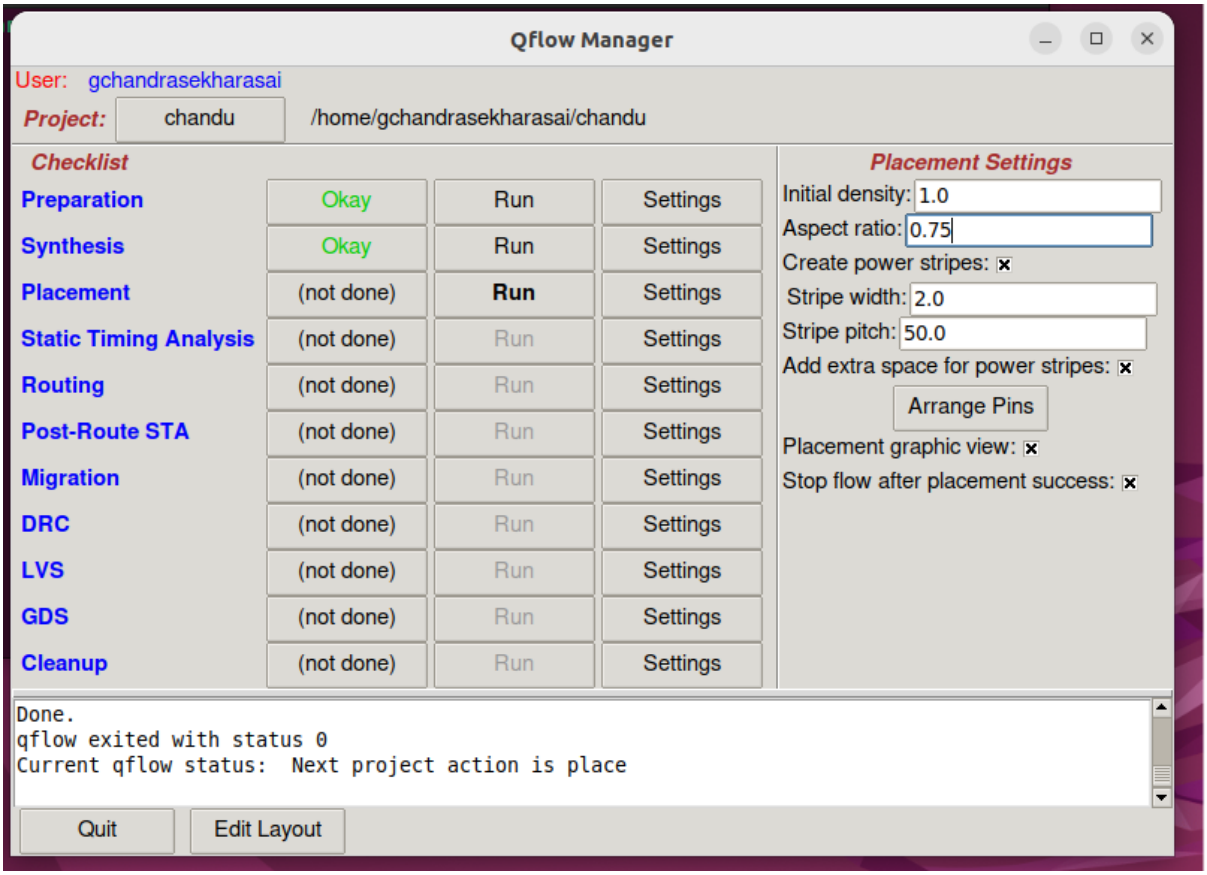
But before running synthesis, make sure your files are saved properly and the project path is correct. Otherwise, errors may occur. You also need to save all your Verilog files in one place and give the folder path in Qflow. After that, click on the synthesis tool. Then wait — if everything goes fine, you'll see OKAY.

Yosys:

Yosys is the main synthesis tool used in the Qflow design setup. It helps convert high-level Verilog (HDL) code into a gate-level design. This means it takes your written logic and turns it into actual circuit structure.

Yosys reads the input design, processes it, and creates an output that shows how the hardware will look (called a gate-level netlist). It supports different file formats like BLIF, which is needed for the next steps like placement, routing, and simulation.

Yosys also helps catch syntax errors and supports features like hierarchy checking and optimizations. It provides extra files (like logs and reports) that can be useful to understand what's going on, debug errors, and improve your design.



Placement

Placement is the process of arranging all the standard cells present in the netlist into the core area of the chip. This is typically handled by EDA tools, which optimize the placement for various design goals.

Types of Placement Techniques

1. Timing-Driven Placement
    - Focuses on minimizing timing delays and meeting timing constraints. o Prioritizes critical path optimization.
2. Congestion-Driven Placement
    - Aims to reduce routing congestion and avoid routing hotspots. o Prioritizes wire length and routing resources.
3. Power-Driven Placement o Targets low power consumption.
    - Places cells to reduce dynamic and leakage power.

Goals of Placement

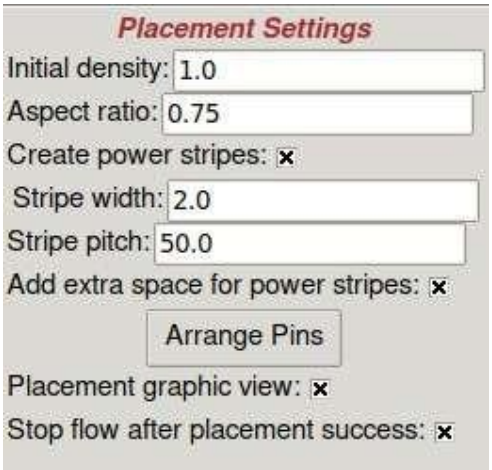
- Timing optimization – ensure setup/hold timing is met.
- Area optimization – efficient use of chip real estate.
- Power optimization – reduce switching and leakage power.
- Minimize congestion – avoid routing difficulties.
- Reduce cell and pin density – avoid overlap or overuse of routing channels.
- Avoid timing DRCs – no design rule violations post placement.

Inputs to Placement

- Netlist – the logical connectivity of the design.
- Physical Libraries (LEF files) – provide standard cell dimensions and pin positions.
- Floorplan DEF file – defines core area, rows, and placement constraints.

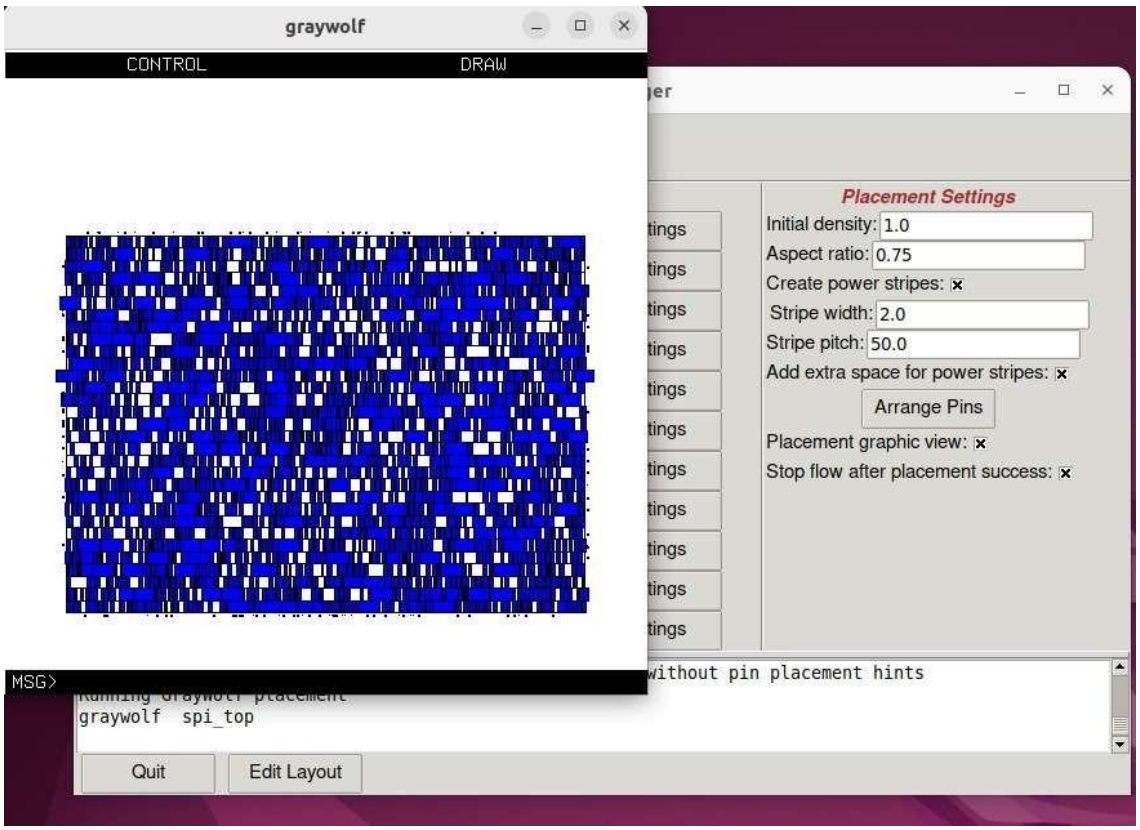
Placement Steps

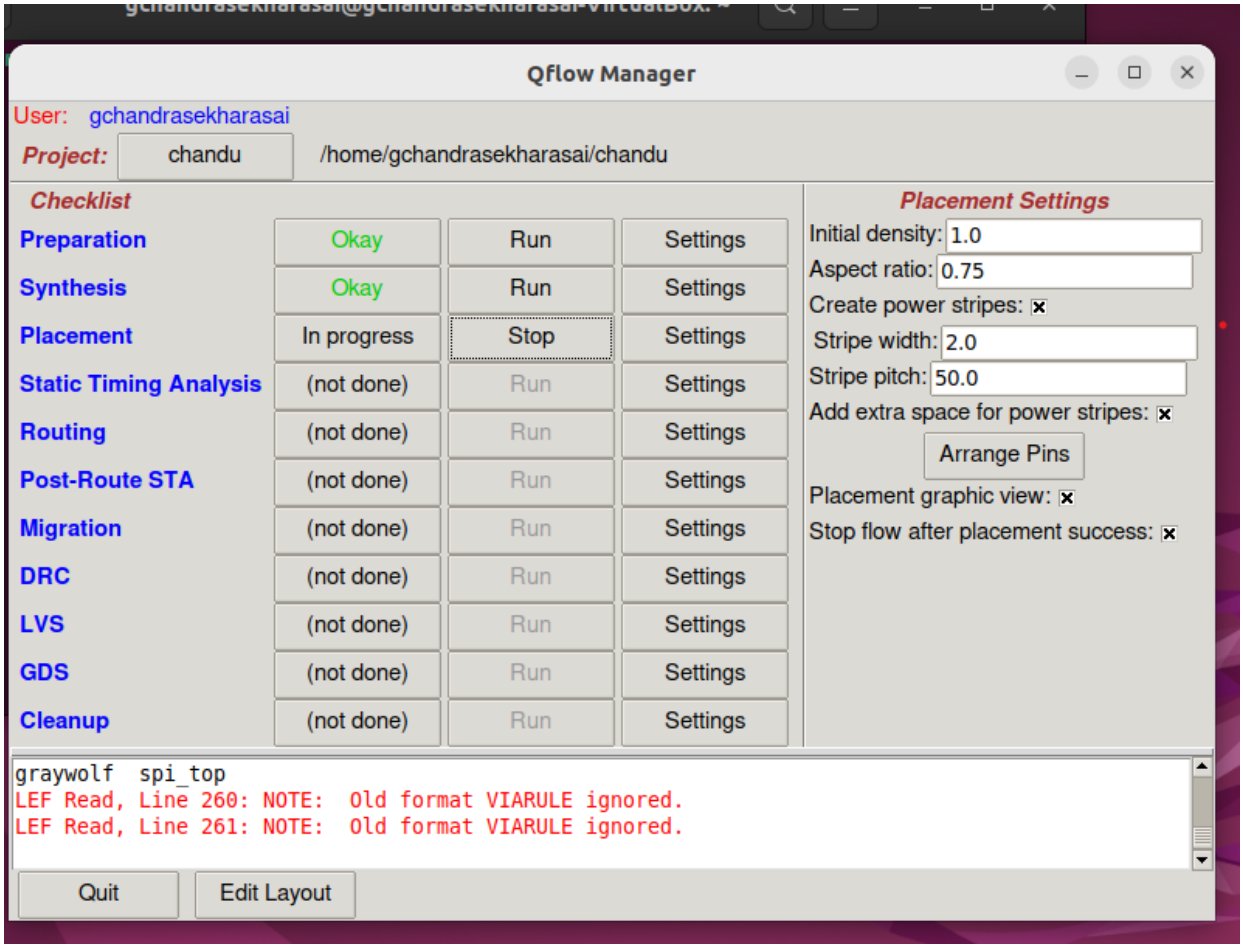
1. Pre-Placement – Prepare the floorplan, insert power rails, and reserve space.
2. Placement – The actual cell placement using algorithms.
3. Post-Placement – Perform optimization, legalize the placement, and check for DRCs.



Graywolf: The Placement Tool in Qflow

In the Qflow toolchain, Graywolf is the placement tool responsible for determining the optimal physical locations of logic cells during the ASIC or FPGA design process. It plays a critical role in the physical design phase by arranging cells strategically within the chip's core area to ensure high performance and efficient resource usage. The primary goal of Graywolf is to minimize signal delays and reduce routing congestion by placing cells in a way that shortens interconnection distances. It uses sophisticated algorithms to balance cell distribution, control cell density, and maintain uniform pin spacing, all while adhering to design constraints such as timing and power. By achieving a balanced and optimized placement, Graywolf helps ensure that the design not only fits within the available silicon area but also operates with minimal timing violations and maximum routing efficiency. This makes it an essential component of the Qflow suite, directly influencing the overall success and performance of the final chip layout.





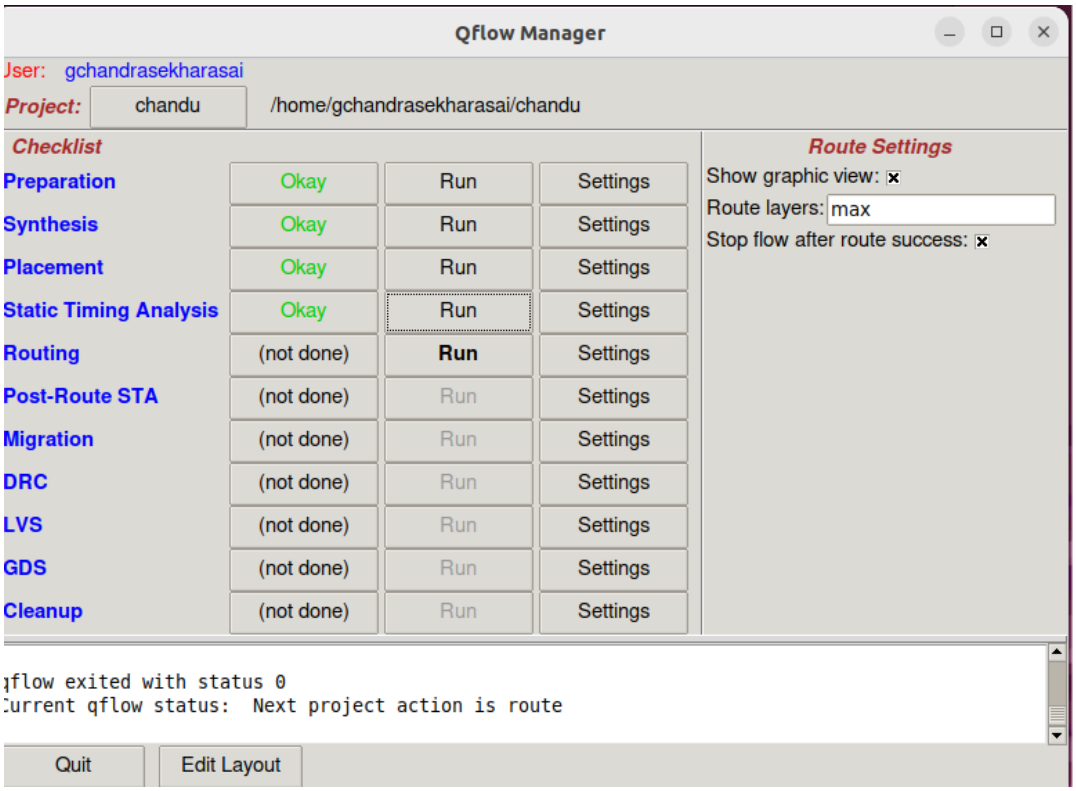
Static Timing Analysis

Static Timing Analysis (STA) is a crucial technique used in digital design to evaluate the timing performance of a circuit without requiring dynamic simulation. Unlike simulation-based methods that circuit behavior under specific input patterns, STA examines the timing characteristics of a design by analyzing path delays (setup and hold) and their propagation delays. It involves checking setup and hold times, clock constraints, and propagation delays to identify worst-case scenarios over our entire design. By evaluating the timing paths through different blocks and components without the need for dynamic input vectors or external stimuli for the clock, STA helps in identifying and eliminating potential timing issues before fabrication.

STA is essential for ensuring that a digital circuit operates reliably at its intended clock speed. It provides a comprehensive assessment of the design's timing margins, allowing designers to determine worstcase timing and improve design

robustness. When designing an integrated circuit, engineers rely on STA to detect timing violations, reduce timing closure time, and verify that the design meets required clock frequencies. STA is particularly effective in large-scale designs where traditional simulation becomes impractical due to time and resource constraints.

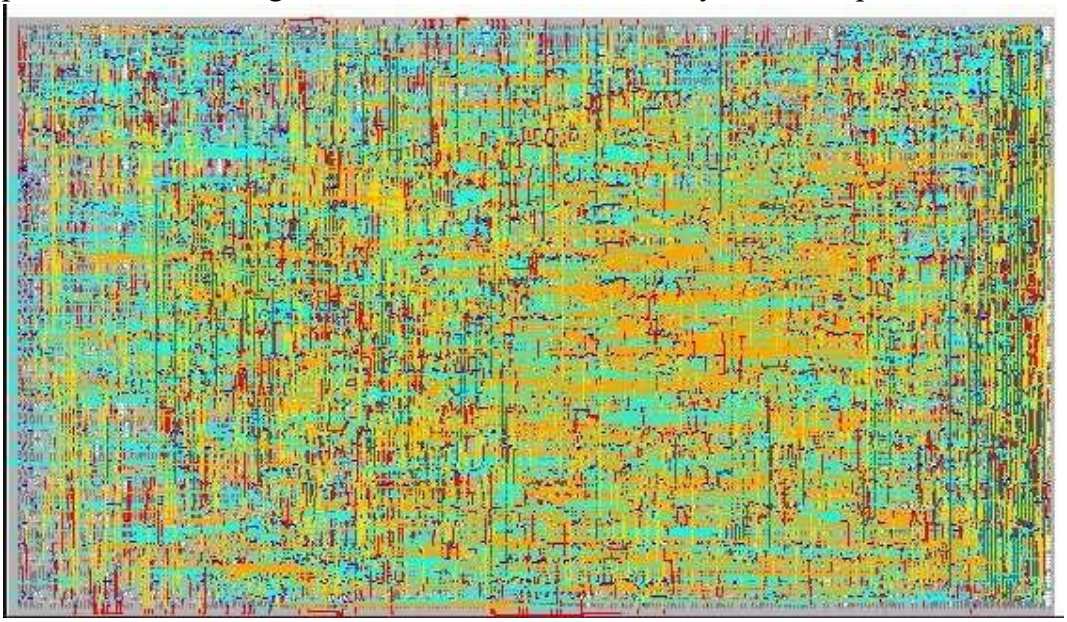
OpenSTA is an open-source Static Timing Analysis tool that is integrated into the OpenROAD project for design automation and optimization. By performing STA on a synthesized netlist and checking against timing constraints, OpenSTA helps ensure timing correctness in the digital design process. By enabling detailed timing analysis during the design cycle, engineers can identify critical paths, optimize performance, and reduce the risk of late-stage design changes. Tools like OpenSTA are important in bringing robust STA capabilities to the design community, enabling both students and professionals to explore STA and the vital role it represents in speed and reliability.

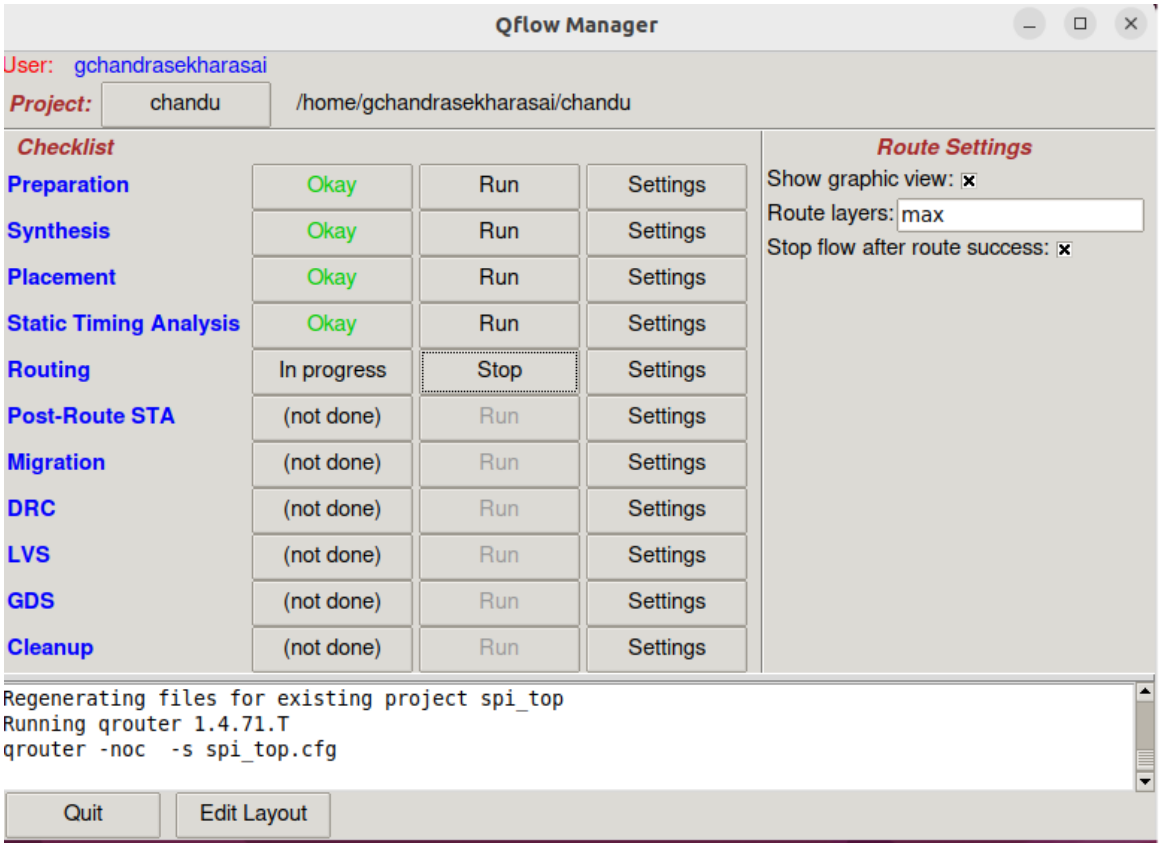


Routing

Routing is the process of establishing electrical connections between the placed standard cells and other components in an ASIC or FPGA layout. Once the placement phase is complete, the routing tool creates the actual metal interconnects that carry signals across the design. The goal of routing is to

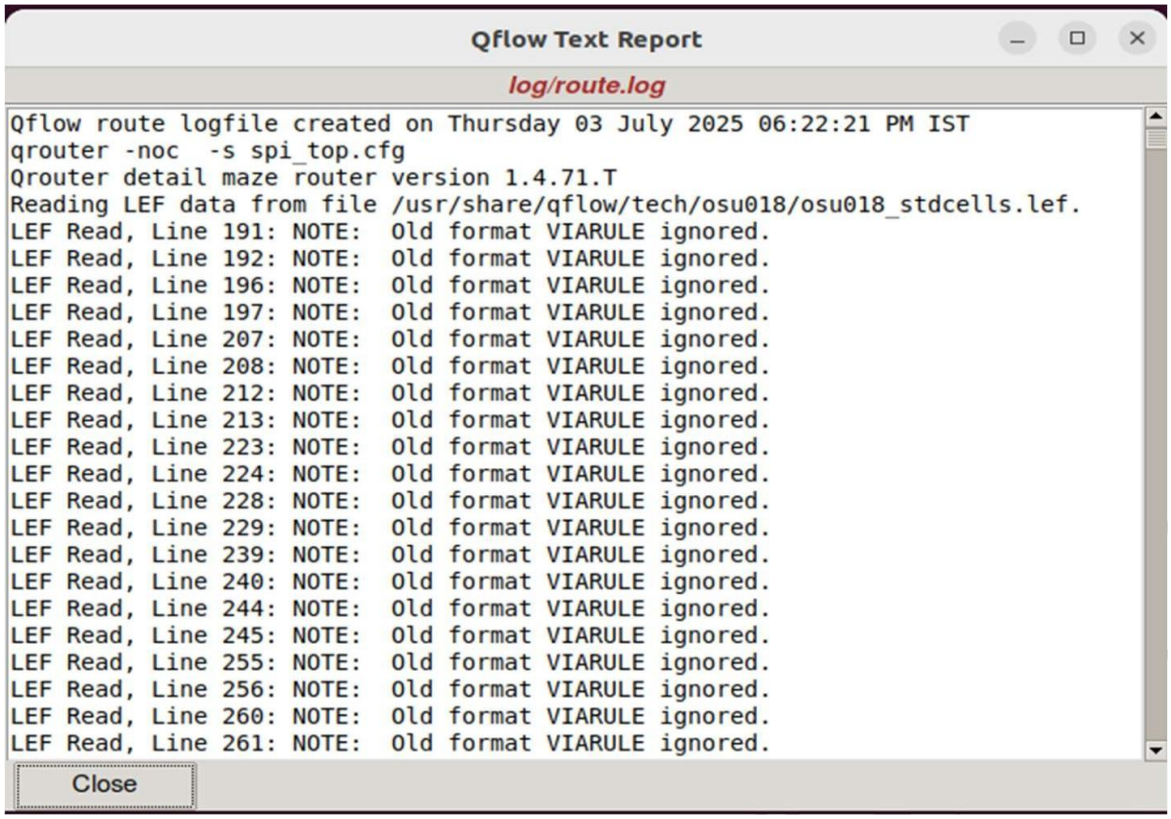
ensure that all nets (signal paths) are fully connected while meeting design constraints such as timing, area, and power. Routing typically proceeds in two stages: global routing, which plans coarse paths for interconnections based on available routing resources, and detailed routing, which finalizes the exact metal layers and wire paths. Efficient routing minimizes wirelength, avoids congestion, reduces parasitic effects, and ensures signal integrity. In the Qflow toolchain, the routing is often handled using the tool QRouter, which takes the DEF file and placement results to generate a legal, design-rule-compliant routing solution. Proper routing is critical because it directly impacts performance, timing closure, and manufacturability of the chip.

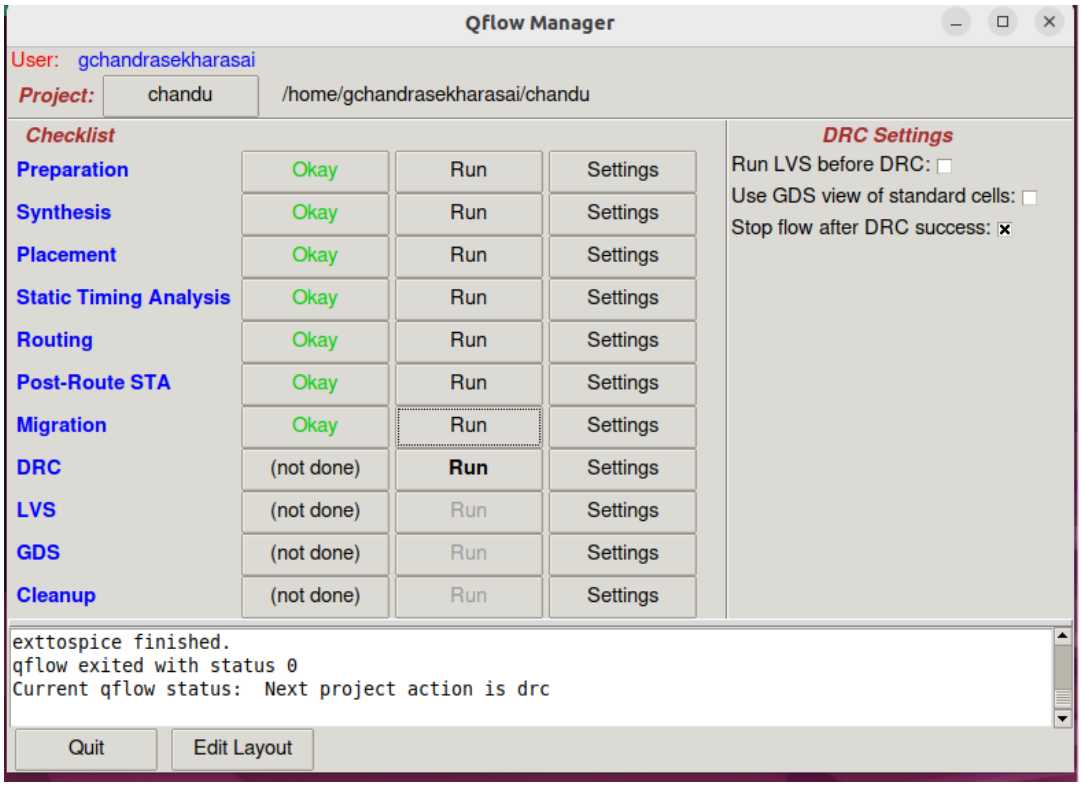




After routing we have a small thing called Post Routing STA

Post-Routing Static Timing Analysis (STA) is the process of verifying the timing performance of a digital circuit after the routing stage has been completed. At this stage, all physical information — including actual wire lengths, parasitics, and delays introduced by interconnects — is available, allowing for accurate timing calculations. The goal of post-routing STA is to ensure that the circuit meets timing requirements under all operating conditions without the need for dynamic simulation. It identifies critical paths and potential violations such as setup or hold time failures, which can affect the functional correctness of the design. In the Qflow design flow, tools like vesta are used to perform STA, analyzing the design based on a standard delay format (SDF) file and checking against the timing constraints specified by the user. This step is essential for timing closure, ensuring that the chip will operate reliably at the intended clock frequency.


Qflow Text Report
log/route.log
Qflow route logfile created on Thursday 03 July 2025 06:22:21 PM IST
qrouter -noc -s spi_top.cfg
Qrouter detail maze router version 1.4.71.T
Reading LEF data from file /usr/share/qflow/tech/osu018/osu018_stdcells.lef.
LEF Read, Line 191: NOTE: Old format VIARULE ignored.
LEF Read, Line 192: NOTE: Old format VIARULE ignored.
LEF Read, Line 196: NOTE: Old format VIARULE ignored.
LEF Read, Line 197: NOTE: Old format VIARULE ignored.
LEF Read, Line 207: NOTE: Old format VIARULE ignored.
LEF Read, Line 208: NOTE: Old format VIARULE ignored.
LEF Read, Line 212: NOTE: Old format VIARULE ignored.
LEF Read, Line 213: NOTE: Old format VIARULE ignored.
LEF Read, Line 223: NOTE: Old format VIARULE ignored.
LEF Read, Line 224: NOTE: Old format VIARULE ignored.
LEF Read, Line 228: NOTE: Old format VIARULE ignored.
LEF Read, Line 229: NOTE: Old format VIARULE ignored.
LEF Read, Line 239: NOTE: Old format VIARULE ignored.
LEF Read, Line 240: NOTE: Old format VIARULE ignored.
LEF Read, Line 244: NOTE: Old format VIARULE ignored.
LEF Read, Line 245: NOTE: Old format VIARULE ignored.
LEF Read, Line 255: NOTE: Old format VIARULE ignored.
LEF Read, Line 256: NOTE: Old format VIARULE ignored.
LEF Read, Line 260: NOTE: Old format VIARULE ignored.
LEF Read, Line 261: NOTE: Old format VIARULE ignored.
Close


Qflow Manager
User: gchandrasekharasai
Project: chandu
/home/gchandrasekharasai/chandu
Checklist
Preparation Okay Run Settings
Synthesis Okay Run Settings
Placement Okay Run Settings
Static Timing Analysis Okay Run Settings
Routing Okay Run Settings
Post-Route STA Okay Run Settings
Migration Okay Run Settings
DRC (not done) Run Settings
LVS (not done) Run Settings
GDS (not done) Run Settings
Cleanup (not done) Run Settings
DRC Settings
Run LVS before DRC:
Use GDS view of standard cells:
Stop flow after DRC success:
exttospice finished.
qflow exited with status 0
Current qflow status: Next project action is drc
Quit
Edit Layout

Design Rule Check (DRC)

Design Rule Check (DRC) is a verification step in the VLSI physical design flow that ensures the layout follows the manufacturing rules set by the foundry. These rules are necessary to guarantee that the chip can be reliably fabricated. DRC checks for violations such as:

- Minimum width of metal layers
- Minimum spacing between wires
- Enclosure and overlap rules for vias and contacts
- Minimum area for metal polygons
- Spacing between wells, n-diffusion, p-diffusion, etc.

When the layout violates any of these rules, DRC flags an error, which must be fixed before tape-out. Tools like Magic are used to perform DRC in open-source flows like Qflow.

Design Rule Check (DRC) is performed using the Magic VLSI layout tool. Magic is one of the most widely used open-source layout editors and rule-checking tools for IC design.

What is Magic?

Magic is a layout tool developed at UC Berkeley, and it is used to draw and edit layouts at the transistor level or standard cell level. It provides powerful features like:

- Layout editing
- DRC (Design Rule Check)
- Extraction
- Simulation support
- LVS (Layout vs. Schematic)

DRC Features in Magic

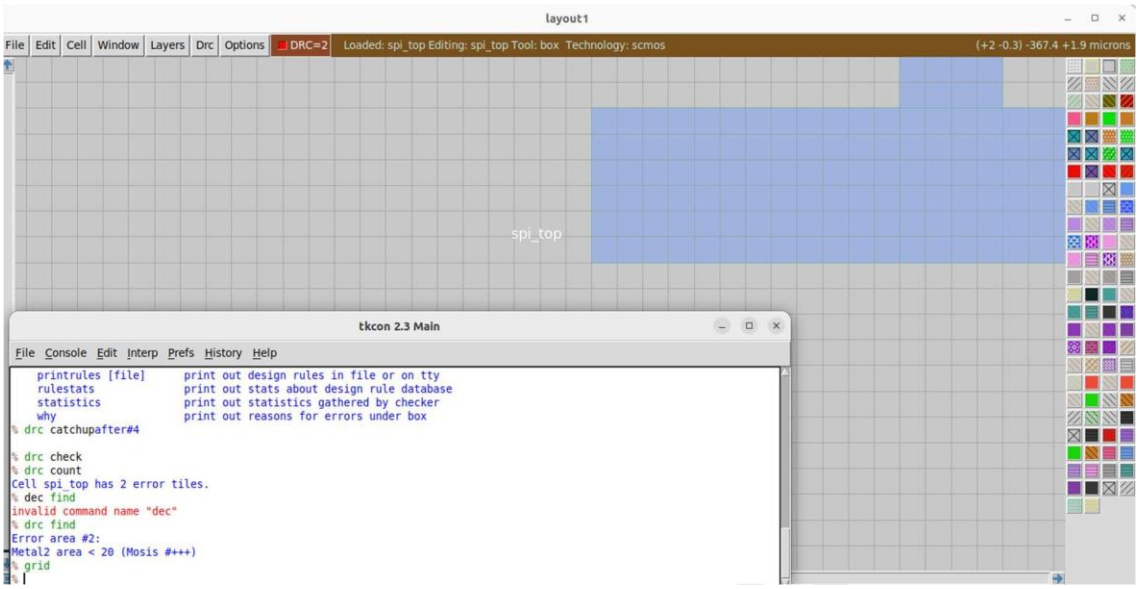
1. Interactive DRC Checking
   As you draw or modify the layout, Magic immediately checks for design rule violations. Errors are shown directly in the layout window (usually in red boxes).

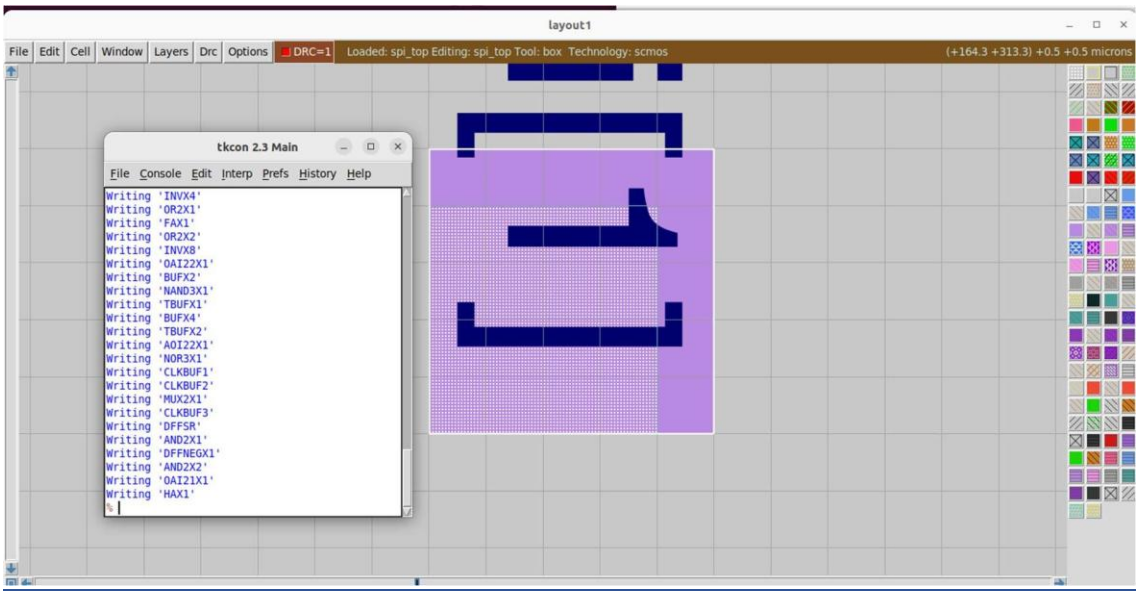
2. Live Feedback
   You can use the tkcon console inside Magic to run DRC commands:

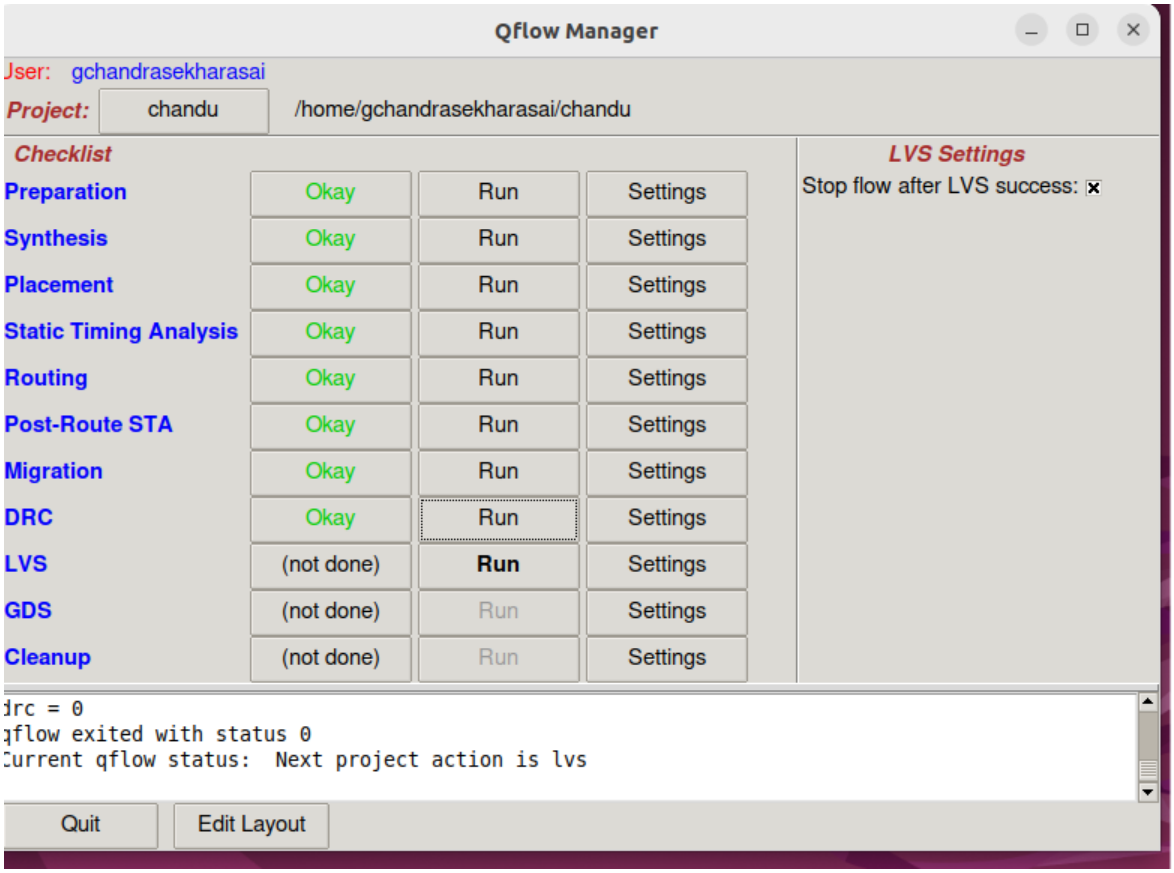
- drc check – Checks for any rule violations. o drc find – Highlights the location of errors.
- drc why – Gives details about the selected DRC error. o drc count – Shows the total number of DRC violations.

3. Error Highlighting
   Magic visually marks error regions with a red outline or bounding box, making it easy to fix them.
4. Grid Support for Area Checks
   The grid command helps you visualize layout sizes. It's especially helpful when fixing errors .




layout1
File Edit Cell Window Layers Drc Options DRC=2 Loaded: spi_top Editing: spi_top Tool: box Technology: scmos (+2 -0.3) -367.4 +1.9 microns
spi_top
tkcon 2.3 Main
File Console Edit Interp Prefs History Help
printrules [file] print out design rules in file or on tty
rulestats print out stats about design rule database
statistics print out statistics gathered by checker
why print out reasons for errors under box
% drc catchupafter#4
% drc check
% drc count
Cell spi_top has 2 error tiles.
% dec find
invalid command name "dec"
% drc find
Error area #2:
Metal2 area < 20 (Mosis #+++)
% grid
%


layout1
File Edit Cell Window Layers Drc Options DRC=1 Loaded: spi_top Editing: spi_top Tool: box Technology: scmos (+164.3 +313.3) +0.5 +0.5 microns
tkcon 2.3 Main
File Console Edit Interp Prefs History Help
Writing 'INVX4'
Writing 'OR2X1'
Writing 'FAX1'
Writing 'OR2X2'
Writing 'INVX8'
Writing 'OAI22X1'
Writing 'BUFX2'
Writing 'NAND3X1'
Writing 'TBUFX1'
Writing 'BUFX4'
Writing 'TBUFX2'
Writing 'AOI22X1'
Writing 'NOR3X1'
Writing 'CLKBUF1'
Writing 'CLKBUF2'
Writing 'MUX2X1'
Writing 'CLKBUF3'
Writing 'DFF5R'
Writing 'AND2X1'
Writing 'DFFNEGX1'
Writing 'AND2X2'
Writing 'OAI21X1'
Writing 'HAX1'
%



LVS

LVS (Layout vs. Schematic) is a critical verification step in the VLSI physical design flow. It ensures that the layout you have drawn is electrically equivalent to the schematic/netlist generated during the RTL-to-GDSII flow.

Purpose of LVS

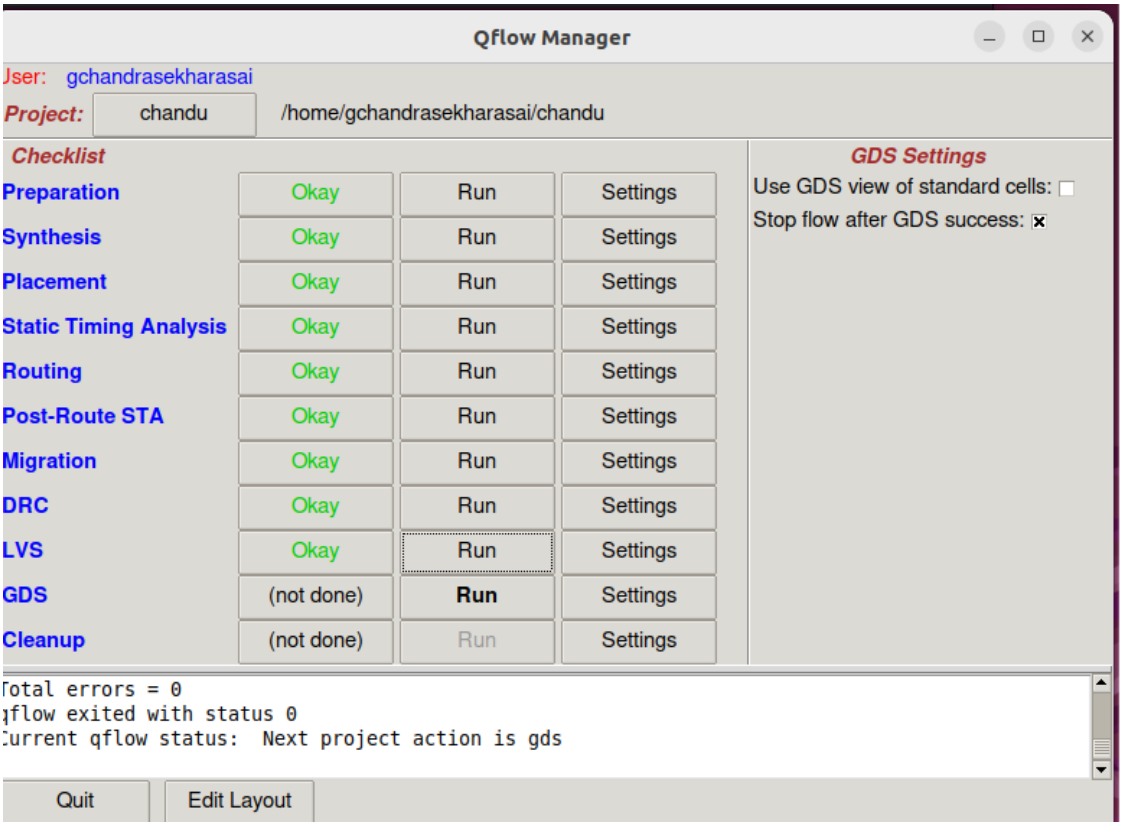
- To check electrical equivalence between the schematic/netlist and the layout.
- To ensure there are no missing or extra components, incorrect connections, or pin mismatches.
- To verify the correctness of the layout before fabrication.

LVS Tool – Netgen

In open-source VLSI workflows like Qflow or OpenLane, Netgen is the tool used to perform LVS checks.

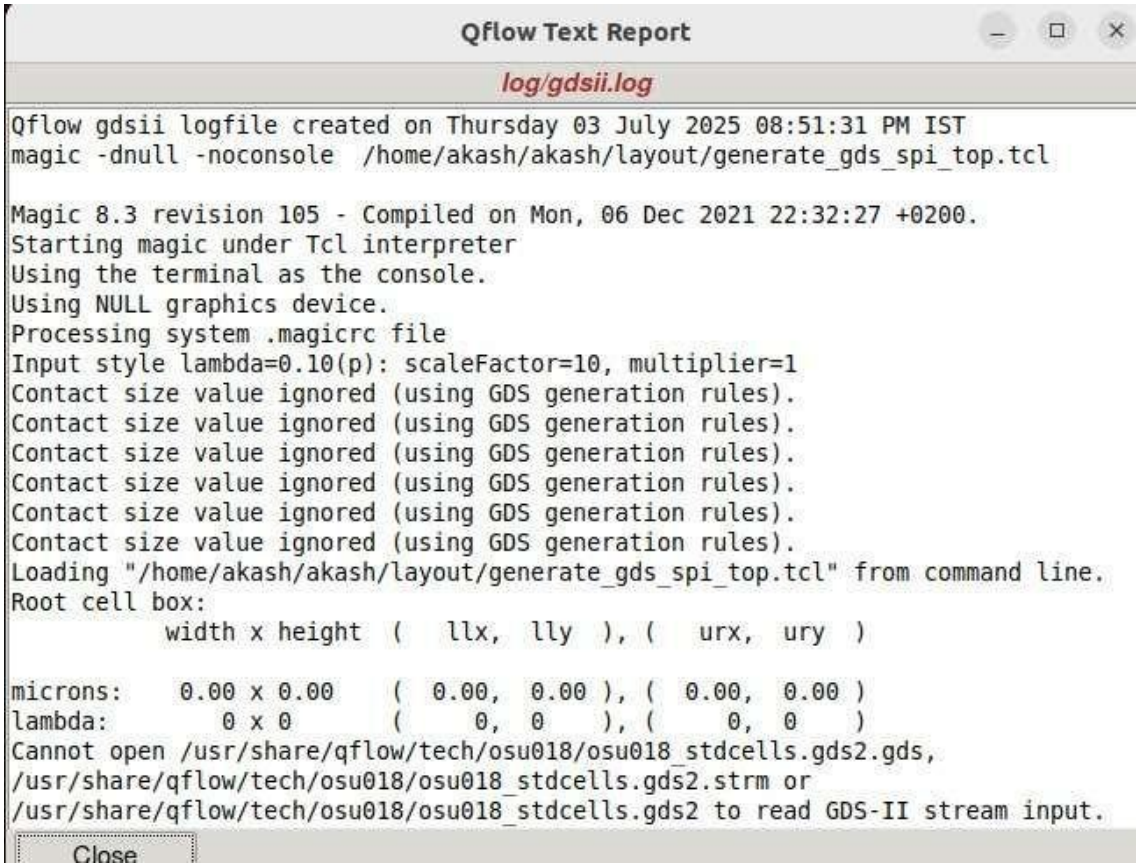
Features of Netgen:

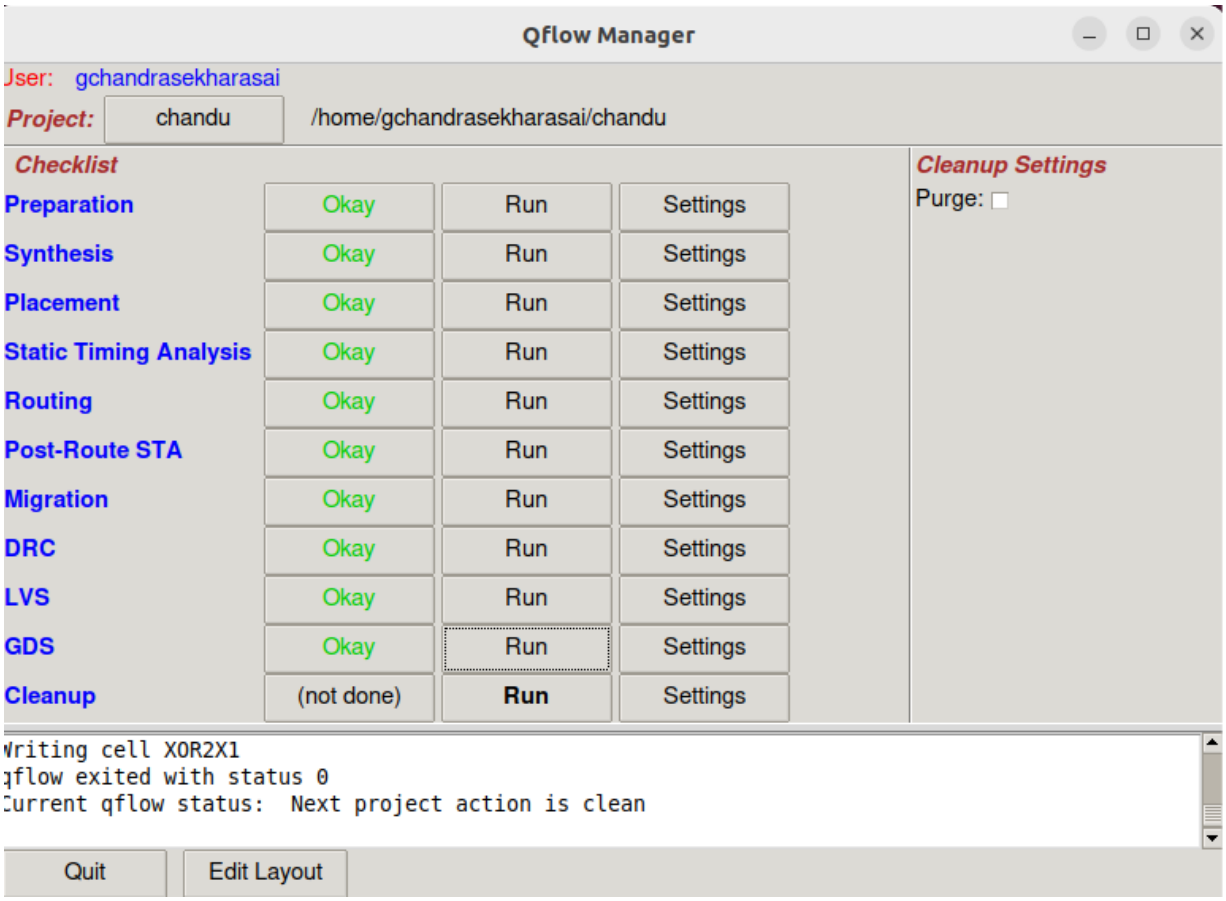
- Compares a SPICE or Verilog netlist (from synthesis) with a SPICE netlist extracted from the layout.
- Reports matches and mismatches in terms of devices, connections, and pins.
- Easy to integrate with Magic for layout extraction.
- Lightweight and widely used in academia and open-source VLSI projects.



GDS

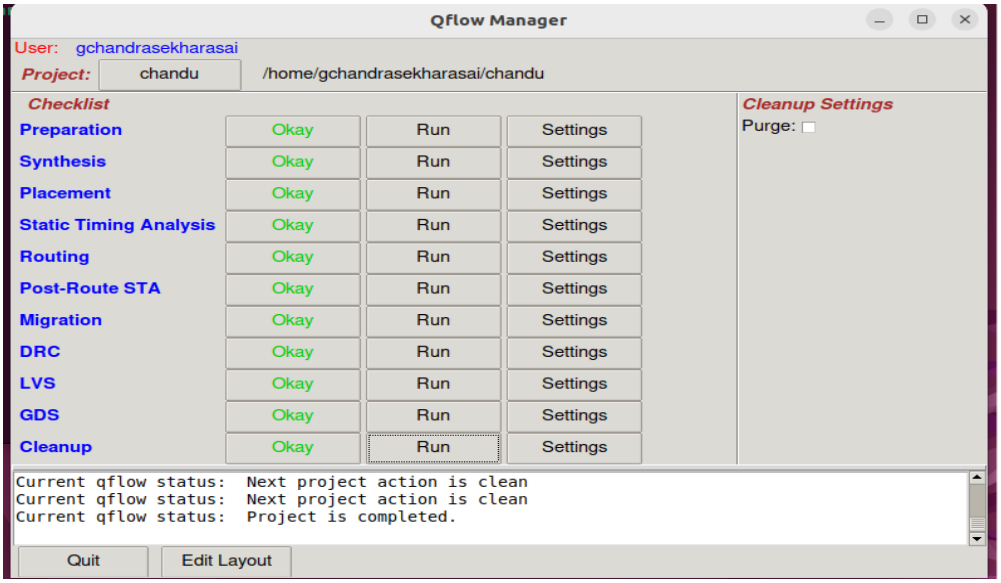
GDS (Graphic Data System), also known as GDSII, is the standard file format used to represent the physical layout of an integrated circuit (IC). It contains all the geometric shapes, layers, and structures needed for chip fabrication. After completing layout design and passing DRC and LVS checks, tools like Magic are used to export the design as a GDS file. This file is then sent to the foundry for manufacturing, making GDS the final and most crucial step in the VLSI physical design flow.


Qflow Text Report
log/gdsii.log
Qflow gdsii logfile created on Thursday 03 July 2025 08:51:31 PM IST
magic -dnull -noconsole /home/akash/akash/layout/generate_gds_spi_top.tcl
Magic 8.3 revision 105 - Compiled on Mon, 06 Dec 2021 22:32:27 +0200.
Starting magic under Tcl interpreter
Using the terminal as the console.
Using NULL graphics device.
Processing system .magicrc file
Input style lambda=0.10(p): scaleFactor=10, multiplier=1
Contact size value ignored (using GDS generation rules).
Contact size value ignored (using GDS generation rules).
Contact size value ignored (using GDS generation rules).
Contact size value ignored (using GDS generation rules).
Contact size value ignored (using GDS generation rules).
Contact size value ignored (using GDS generation rules).
Loading "/home/akash/akash/layout/generate_gds_spi_top.tcl" from command line.
Root cell box:
width x height (llx, lly), (urx, ury)
microns: 0.00 x 0.00 (0.00, 0.00), (0.00, 0.00)
lambda: 0 x 0 (0, 0), (0, 0)
Cannot open /usr/share/qflow/tech/osu018/osu018_stdcells.gds2.gds,
/usr/share/qflow/tech/osu018/osu018_stdcells.gds2.strm or
/usr/share/qflow/tech/osu018/osu018_stdcells.gds2 to read GDS-II stream input.
Close


Qflow Manager
User: gchandrasekharasai
Project: chandu
/home/gchandrasekharasai/chandu
Checklist
Cleanup Settings
Purge:
Preparation Okay Run Settings
Synthesis Okay Run Settings
Placement Okay Run Settings
Static Timing Analysis Okay Run Settings
Routing Okay Run Settings
Post-Route STA Okay Run Settings
Migration Okay Run Settings
DRC Okay Run Settings
LVS Okay Run Settings
GDS Okay Run Settings
Cleanup (not done) Run Settings
Writing cell XOR2X1
qflow exited with status 0
Current qflow status: Next project action is clean
Quit
Edit Layout

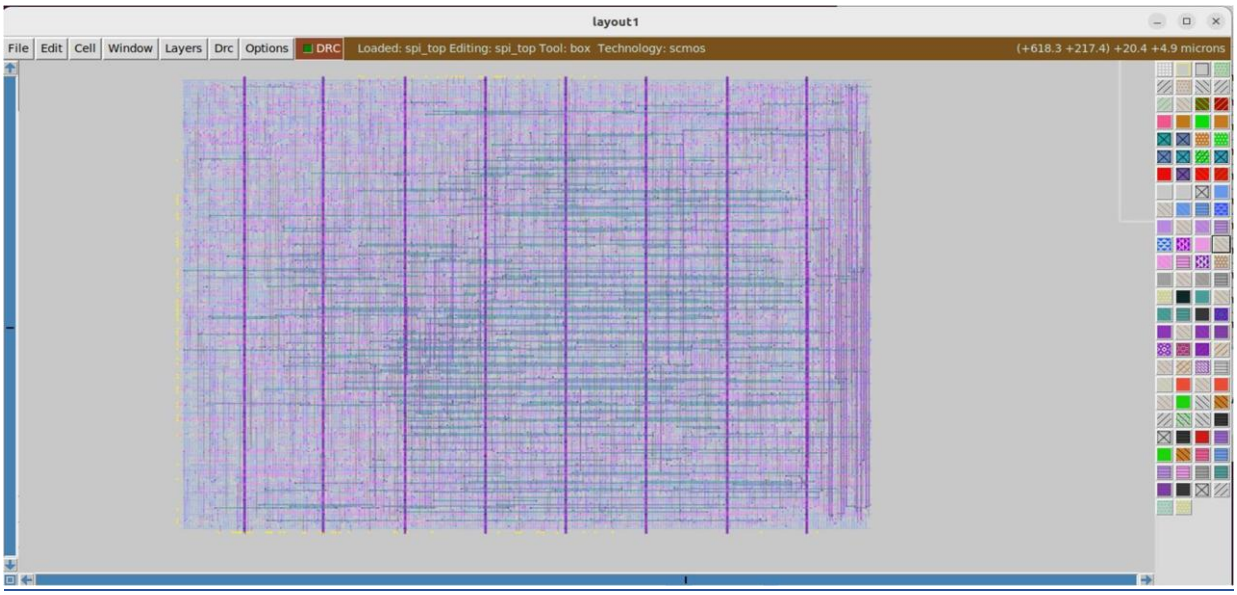
AFTER THIS WE HAVE A SMALL STEP CALLED CLEAN UP

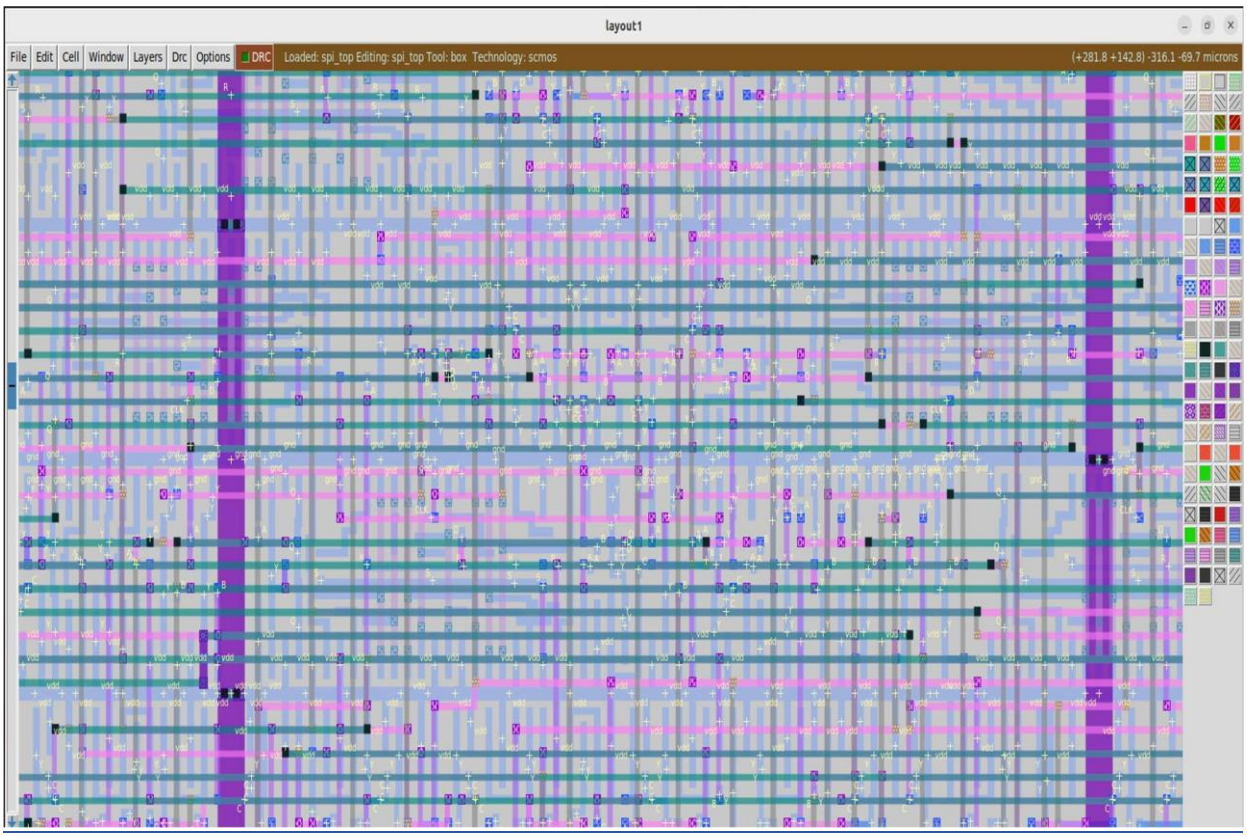
refers to the final step in the digital flow where temporary, intermediate, or unnecessary files generated during the synthesis, placement, routing, and verification processes are deleted. This helps in freeing up disk space and ensuring that only the essential output files—like GDS, netlists, or reports—are retained. Tools like Qflow often include a cleanup script to simplify this process. It's typically run after completing all design steps and verifying that the final results are

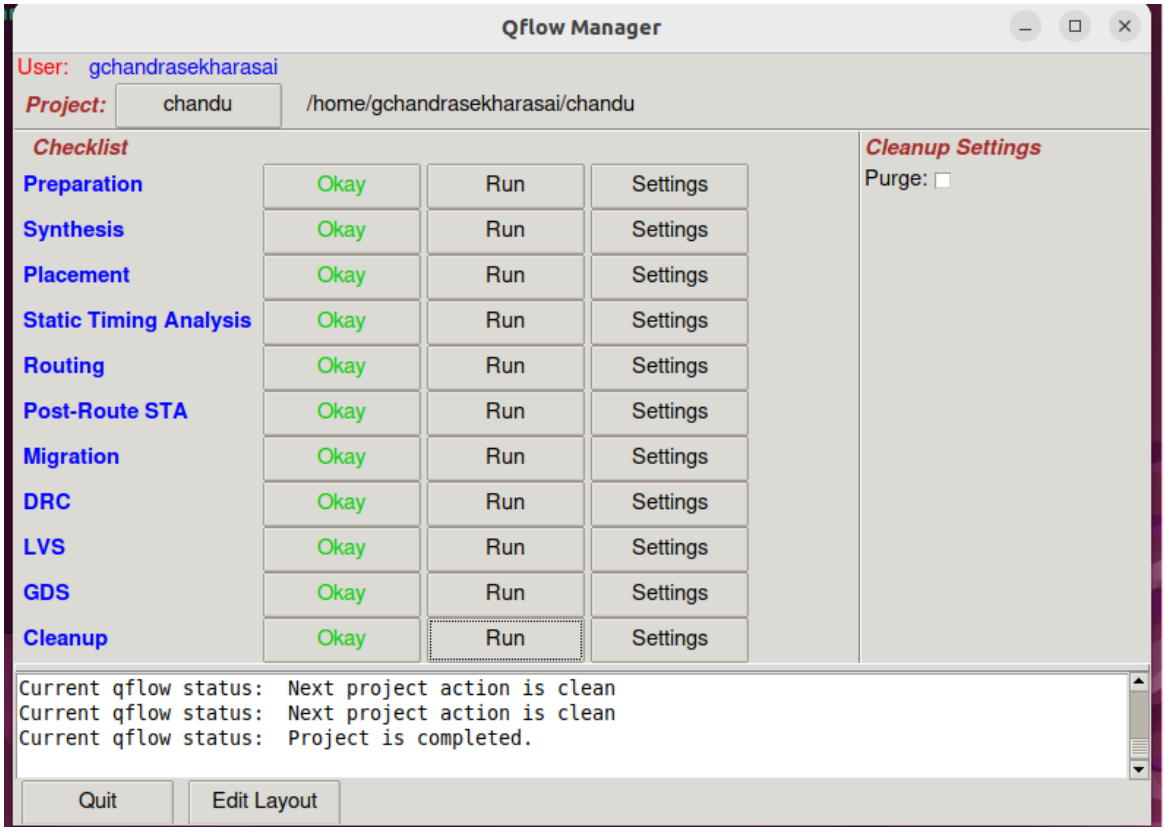


correct.

Final Output







Finally, the Qflow tool generates the GDS file (with .gds extension), which is the standard format used to represent the physical layout of an integrated circuit. This file can be sent to the semiconductor foundry for fabrication.

Thus, the physical design of the Serial Peripheral Interface (SPI) has been successfully implemented using Qflow.

A Project Report by

G.Chandrasekhara sai

03-07-2025